

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – TECHNICAL PRESENTATION

Course Code:	CSA0620	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Technical Presentation
Weightage:	10% of CIA	Total Marks:	100 (1 Question × 100 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	K.Deepika	Reg. No.:	192371042
Section / Batch:	CSE-4th year	Date:	23/07/2026

Instructions to Students

- Answer the given question through a technical presentation. Total Marks: 100.
- Clearly define the problem and explain the concept of algorithm growth functions.
- Present comparative analysis using appropriate representations such as graphs, tables, or charts.
- Use suitable tools (PowerPoint, visualization tools, or software) to support your explanation.
- Ensure technical accuracy, logical flow, and proper interpretation of results.
- Maintain clarity in communication, proper slide organization, and professional delivery.
- Time management, confidence, and audience engagement will be considered during evaluation.

Question Type	Focus Area	Questions	Marks
Technical Presentation	Analyze and compare growth functions using mathematical techniques and asymptotic concepts and with graphical and visual interpretation	Q1	100
GRAND TOTAL:		100 Marks	

SECTION A – Technical Presentation

Q1.Master Theorem Application for Quadratic Work Function

Consider the recurrence relation $T(n) = 2T(n/2) + n^2$. Analyze the problem using the Master Theorem and determine the time complexity. Clearly identify parameters and compare $f(n)$ with $n^{\log_b a}$. Justify the selected case and explain each step. Use recursion tree diagrams if required and interpret the result clearly.

Code of Conduct

I K.DEEPIKA (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K.Deepika(192371042)

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Marks Evaluation:

Question No.	Technical Content Accuracy (30%)	Problem Identification (25%)	Use of Tools (20%)	Communication (15%)	Professionalism (10%)	Total Marks (100)
Q1	4	4	4	2	3	90
Overall Total (100)						90

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____

Date: _____

Date: _____

Date: _____

DESIGN & ANALYSIS OF ALGORITHMS

Master Theorem Application for a Quadratic Work Function

Analyzing $T(n) = 2T(n/2) + n^2$ using the Master Theorem

$$T(n) = 2T(n/2) + n^2$$

Problem Statement

We are given the recurrence relation:

$$T(n) = 2 T(n/2) + n^2$$

- The recurrence divides a problem of size n into 2 sub-problems, each of size $n/2$.
- An additional cost of n^2 is spent outside the recursive calls (the “combine” or “extra work” step).
- Goal: determine the asymptotic (Big- Θ) time complexity $T(n)$ using the Master Theorem, rather than unrolling the recurrence by hand.

?

What we need to identify

a

number of recursive sub-problems

b

factor by which n shrinks each call

f(n)

cost of work done outside recursion

$n^{\log_b a}$

the “watershed” function to compare $f(n)$ against

Algorithm Overview

This recurrence models any Divide-and-Conquer algorithm whose “combine” step costs quadratic time — for example, a merge routine that compares every element of one half against every element of the other.

1

DIVIDE

Split the input array of size n into two halves of size $n/2$ each.

2

CONQUER

Recursively solve each half by calling the same procedure — giving the $2T(n/2)$ term.

3

COMBINE

Merge the two solved halves using a nested (pairwise) $O(n^2)$ procedure — giving the $+n^2$ term.

Net effect: work shrinks per sub-problem, but the quadratic combine step at the top level turns out to dominate the total — as shown in the Complexity Analysis slide.

Algorithm / Pseudocode

```
SOLVE(A, low, high):
    if low >= high:
        return // base case,  $O(1)$ 

    mid = (low + high) / 2

    SOLVE(A, low, mid) //  $T(n/2)$ 
    SOLVE(A, mid+1, high) //  $T(n/2)$ 

    COMBINE(A, low, mid, high) //  $\Theta(n^2)$ 

COMBINE(A, low, mid, high):
    for i = low to mid:
        for j = mid+1 to high:
            // pairwise cross-comparison / merge work
            process(A[i], A[j])
```

SOLVE calls itself twice

on halves of size $n/2 \rightarrow$ the " $2T(n/2)$ " term in the recurrence.

COMBINE runs a double loop

of length $\sim n/2$ each $\rightarrow \Theta(n^2)$ work — the " $+n^2$ " term.

Base case

A single element ($low \geq high$) needs $O(1)$ time and stops the recursion.

Working Example (Dry Run)

Trace the recursion tree for $n = 8$, level by level, and sum the work actually done at each depth.



Level i	Sub-problems (2^i)	Size per problem ($n/2^i$)	Work per node	Total work at level
0	1	8	$8^2 = 64$	$1 \times 64 = 64$
1	2	4	$4^2 = 16$	$2 \times 16 = 32$
2	4	2	$2^2 = 4$	$4 \times 4 = 16$
3	8	1	$1^2 = 1$	$8 \times 1 = 8$

Total work = $64 + 32 + 16 + 8 = 120$

Compare to $n^2 = 8^2 = 64$: the total (120) is of the same order as n^2 — i.e., a small constant multiple of n^2 , not asymptotically larger.

Key observation

Work at each level is a geometric series that shrinks by half going down the tree — so the root level ($n^2 = 64$) contributes the majority of the total cost.

Complexity Analysis — Master Theorem

Step 1 — Identify parameters

$a = 2$ (recursive calls)
 $b = 2$ (input shrink factor)
 $f(n) = n^2$ (extra work per call)

Step 2 — Compute the watershed function

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

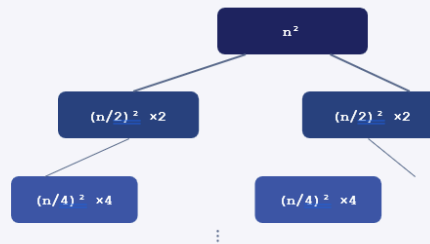
Step 3 — Compare $f(n)$ with $n^{\log_b a}$

$$f(n) = n^2 \quad \text{vs.} \quad n^{\log_b a} = n^1$$

Since $n^2 = \Omega(n^1 \cdot \epsilon)$ for $\epsilon = 1$, $f(n)$ grows polynomially faster $\rightarrow$ Case 3 condition satisfied.

Regularity check: $a \cdot f(n/b) \leq c \cdot f(n)$
 $2 \cdot (n/2)^2 = n^2/2 = 0.5 \cdot f(n) \rightarrow$ holds for $c = 0.5 < 1$ ✓

Recursion Tree



Level i total = $n^2 \cdot (1/2)^i \rightarrow$ geometric series

$$\text{Sum} \approx n^2 \cdot (1 + 1/2 + 1/4 + \dots) = 2n^2 \rightarrow T(n) = \Theta(n^2)$$

Comparative Analysis

The three Master Theorem cases, compared side by side with our recurrence.

Case	Condition	Example recurrence	Result
1	$f(n) = O(n^{\log_b a - \epsilon})$	$T(n) = 8T(n/2) + n^2$ ($\log_b a = 3$)	$\Theta(n^3)$ recursion dominates
2	$f(n) = O(n^{\log_b a})$	$T(n) = 2T(n/2) + n$ ($\log_b a = 1$) — e.g. Merge Sort	$\Theta(n \log n)$ balanced
3 ★	$f(n) = O(n^{\log_b a + \epsilon})$, regularity holds	$T(n) = 2T(n/2) + n^2$ ($\log_b a = 1$) — OUR CASE	$\Theta(n^2)$ extra work dominates

Insight: even though our algorithm halves the problem at each step, the $\Theta(n^2)$ combine step never gets any cheaper relative to the shrinking recursion — so it behaves asymptotically like a single-pass $O(n^2)$ algorithm with no divide-and-conquer speed-up.

Applications

Recurrences of the form $T(n) = 2T(n/2) + n^2$ appear whenever a divide-and-conquer design has a quadratic “stitching” step.

A

Naive Merge Variants

Merge routines that pairwise-compare cross elements between two sorted halves instead of a linear scan.

B

Brute-force Combine Steps

Divide-and-conquer closest-pair or matching algorithms whose merge phase checks all cross-pairs (before optimizations like strip-checking).

C

Naive Matrix / Convolution Blending

Divide-and-conquer image or signal processing where two blended halves are combined via a full nested-loop convolution.

D

Teaching Tool for Master Theorem

Canonical textbook example to illustrate Case 3 — showing when the non-recursive part of the work provably dominates.

Results & Discussion

RESULT

$$\Theta(n^2)$$

Master Theorem — Case 3
($f(n)$ dominates)

Confirmed independently by
the recursion-tree summation
(Slide 5 & 6): $\approx 2n^2$

1

The extra n^2 work at the root is asymptotically as large as the sum of all work at every other level combined — the recursion contributes only a constant-factor overhead.

2

Compared with Merge Sort ($\Theta(n \log n)$), adding a quadratic combine step erases the benefit of dividing the problem — the algorithm is no faster, order-wise, than a single $\Theta(n^2)$ pass.

3

This demonstrates why algorithm designers try to keep the “combine” step linear or near-linear (as in Merge Sort or Karatsuba multiplication) — a cheap combine step is what lets divide-and-conquer pay off.

Conclusion & References

Conclusion

- For $T(n) = 2T(n/2) + n^2$: $a = 2$, $b = 2$, $f(n) = n^2$.
- Since $n^2 = \Omega(n^{\log_b a + \epsilon})$ and the regularity condition holds, Case 3 of the Master Theorem applies.
- Therefore $T(n) = \Theta(n^2)$ — confirmed by direct recursion-tree summation.
- The result generalizes: whenever the non-recursive cost strictly dominates the recursive cost, that cost sets the overall running time.

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. — Introduction to Algorithms (CLRS), Chapter on “Divide-and-Conquer / The Master Method.”
- Kleinberg, J., & Tardos, É. — Algorithm Design, section on solving recurrences.
- Sedgewick, R., & Wayne, K. — Algorithms, on recurrence analysis for divide-and-conquer methods.
- Standard DAA / GATE course notes on the Master Theorem and recursion-tree method.

GEOTAG PHOTO

